

BASIC OPERATIONS USING NUMPY ARRAYS

SOURCE CODE

a) Create NumPy arrays from Python Data Structures, Intrinsic NumPy objects and Random Functions

Property	Program	Output
Get the data type of an array object	<pre>import numpy as np arr = np.array([1, 2, 3, 4]) print(arr.dtype)</pre>	int64
Get the data type of an array containing strings	<pre>import numpy as np arr = np.array(['apple', 'banana', 'cherry']) print(arr.dtype)</pre>	<U6
Create an array with data type string	<pre>import numpy as np arr = np.array([1, 2, 3, 4], dtype='S') print(arr) print(arr.dtype)</pre>	[b'1' b'2' b'3' b'4'] S1
Create an array with data type 4 bytes integer	<pre>import numpy as np arr = np.array([1, 2, 3, 4], dtype='i4') print(arr) print(arr.dtype)</pre>	[1 2 3 4] int32
Change data type from float to integer using 'i' as parameter value	<pre>import numpy as np arr = np.array([1.1, 2.1, 3.1]) newarr = arr.astype('i') print(newarr) print(newarr.dtype)</pre>	[1 2 3] int32
Change data type from float to integer using int as parameter value	<pre>import numpy as np arr = np.array([1.1, 2.1, 3.1]) newarr = arr.astype(int) print(newarr) print(newarr.dtype)</pre>	[1 2 3] int64
Change data type from integer to boolean	<pre>import numpy as np arr = np.array([1, 0, 3]) newarr = arr.astype(bool) print(newarr) print(newarr.dtype)</pre>	[True False True] bool
Demonstrate the working of array()	<pre>import array arr = array.array('i', [1, 2, 3]) print("The new created array is : ", end="") for i in range(0, 3): print(arr[i], end=" ") print("\r")</pre>	The new created array is : 1 2 3 ■
Intrinsic NumPy array creation: arange()	<pre>import numpy as np arr = np.arange(10) print(arr)</pre>	[0 1 2 3 4 5 6 7 8 9]
arange() with start, stop and dtype	<pre>import numpy as np arr = np.arange(2, 10, dtype=float) print(arr)</pre>	[2. 3. 4. 5. 6. 7. 8. 9.]
arange() with a float step	<pre>import numpy as np arr = np.arange(2, 3, 0.1) print(arr)</pre>	[2. 2.1 2.2 2.3 2.4 2.5 2.6 2.7 2.8 2.9]
Other intrinsic creators: zeros, ones, linspace, eye	<pre>import numpy as np print(np.zeros((2, 3))) print(np.ones((2, 2))) print(np.linspace(0, 1, 5)) print(np.eye(3))</pre>	[[0. 0. 0.] [0. 0. 0.] [[1. 1.] [1. 1.] [0. 0.25 0.5 0.75 1.] [[1. 0. 0.] [0. 1. 0.] [0. 0. 1.]
Generate a random integer from 0 to 100	<pre>from numpy import random x = random.randint(100) print(x)</pre>	46
Generate a random float from 0 to 1	<pre>from numpy import random x = random.rand() print(x)</pre>	0.7820722809872843

Property	Program	Output
Generate a 1-D array containing 5 random integers from 0 to 100	<pre>from numpy import random x = random.randint(100, size=(5)) print(x)</pre>	[93 13 72 15 89]
Generate a 2-D array with 3 rows, each row containing 5 random integers from 0 to 100	<pre>from numpy import random x = random.randint(100, size=(3, 5)) print(x)</pre>	[[56 73 2 59 23] [27 59 0 48 9] [46 13 98 45 12]]
Generate a 1-D array containing 5 random floats	<pre>from numpy import random x = random.rand(5) print(x)</pre>	[0.29881738 0.27964112 0.27426994 0.62182981 0.59702601]

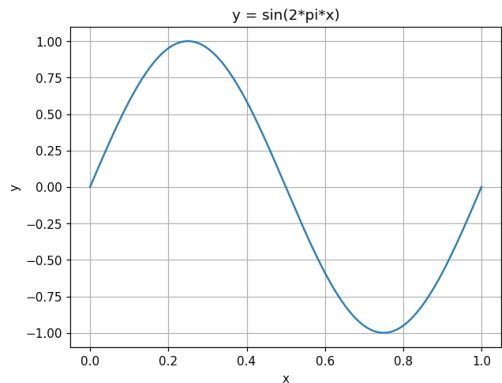
b) Manipulation of NumPy arrays - Indexing, Slicing, Reshaping, Joining and Splitting

Property	Program	Output
Indexing: get the first element	<pre>import numpy as np arr = np.array([1, 2, 3, 4]) print(arr[0])</pre>	1
Indexing: get the third and fourth elements and add them	<pre>import numpy as np arr = np.array([1, 2, 3, 4]) print(arr[2] + arr[3])</pre>	7
Slicing: elements from index 1 to index 5	<pre>import numpy as np arr = np.array([1, 2, 3, 4, 5, 6, 7]) print(arr[1:5])</pre>	[2 3 4 5]
Slicing: from the beginning to index 4	<pre>import numpy as np arr = np.array([1, 2, 3, 4, 5, 6, 7]) print(arr[:4])</pre>	[1 2 3 4]
Reshaping: 1-D array with 12 elements into a 2-D array	<pre>import numpy as np arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]) newarr = arr.reshape(4, 3) print(newarr)</pre>	[[1 2 3] [4 5 6] [7 8 9] [10 11 12]]
Reshaping: 1-D array with 12 elements into a 3-D array	<pre>import numpy as np arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]) newarr = arr.reshape(2, 3, 2) print(newarr)</pre>	[[[1 2] [3 4] [5 6]] [[7 8] [9 10] [11 12]]]
Reshaping: 1-D array with 8 elements to a 3-D array (unknown dimension -1)	<pre>import numpy as np arr = np.array([1, 2, 3, 4, 5, 6, 7, 8]) newarr = arr.reshape(2, 2, -1) print(newarr)</pre>	[[[1 2] [3 4]] [[5 6] [7 8]]]
Reshaping: flatten the array into 1-D	<pre>import numpy as np arr = np.array([[1, 2, 3], [4, 5, 6]]) newarr = arr.reshape(-1) print(newarr)</pre>	[1 2 3 4 5 6]
Joining: join two arrays	<pre>import numpy as np arr1 = np.array([1, 2, 3]) arr2 = np.array([4, 5, 6]) arr = np.concatenate((arr1, arr2)) print(arr)</pre>	[1 2 3 4 5 6]
Joining: two 2-D arrays along axis=1	<pre>import numpy as np arr1 = np.array([[1, 2], [3, 4]]) arr2 = np.array([[5, 6], [7, 8]]) arr = np.concatenate((arr1, arr2), axis=1) print(arr)</pre>	[[1 2 5 6] [3 4 7 8]]
Joining arrays using stack functions	<pre>import numpy as np arr1 = np.array([1, 2, 3]) arr2 = np.array([4, 5, 6]) arr = np.stack((arr1, arr2), axis=1) print(arr)</pre>	[[1 4] [2 5] [3 6]]

Property	Program	Output
Splitting: split the array in 3 parts	<pre>import numpy as np arr = np.array([1, 2, 3, 4, 5, 6]) newarr = np.array_split(arr, 3) print(newarr)</pre>	<pre>[array([1, 2]), array([3, 4]), array([5, 6])]</pre>
Splitting: split the array in 4 parts	<pre>import numpy as np arr = np.array([1, 2, 3, 4, 5, 6]) newarr = np.array_split(arr, 4) print(newarr)</pre>	<pre>[array([1, 2]), array([3, 4]), array([5]), array([6])]</pre>
Splitting: access the split arrays	<pre>import numpy as np arr = np.array([1, 2, 3, 4, 5, 6]) newarr = np.array_split(arr, 3) print(newarr[0]) print(newarr[1]) print(newarr[2])</pre>	<pre>[1 2] [3 4] [5 6]</pre>
Splitting: 2-D array into three 2-D arrays	<pre>import numpy as np arr = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]]) newarr = np.array_split(arr, 3) print(newarr)</pre>	<pre>[array([[1, 2], [3, 4]]), array([[5, 6], [7, 8]]), array([[9, 10], [11, 12]])]</pre>
Splitting: 2-D array into three 2-D arrays along axis=1	<pre>import numpy as np arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12], [13, 14, 15], [16, 17, 18]]) newarr = np.array_split(arr, 3, axis=1) print(newarr)</pre>	<pre>[array([[1], [4], [7], [10], [13], [16]]), array([[2], [5], [8], [11], [14], [17]]), array([[3], [6], [9], [12], [15], [18]])]</pre>

c) Computation on NumPy arrays using Universal Functions and Mathematical methods

Property	Program	Output
Create your own ufunc for addition	<pre>import numpy as np def myadd(x, y): return x + y myadd = np.frompyfunc(myadd, 2, 1) print(myadd([1, 2, 3, 4], [5, 6, 7, 8]))</pre>	<pre>[6 8 10 12]</pre>
Check if a function is a ufunc	<pre>import numpy as np print(type(np.add))</pre>	<pre><class 'numpy.ufunc'></pre>
Use an if statement to check if the function is a ufunc	<pre>import numpy as np if type(np.add) == np.ufunc: print('add is ufunc') else: print('add is not ufunc')</pre>	<pre>add is ufunc</pre>
Compute the values sin(2*pi*x)	<pre>import numpy as np x = np.arange(0, 1.25, 0.25) print(x) y = np.sin(2 * np.pi * x) print(y)</pre>	<pre>[0. 0.25 0.5 0.75 1.] [0. 1. 0. -1. -0.]</pre>
Convert all values in an array to radians	<pre>import numpy as np arr = np.array([90, 180, 270, 360]) x = np.deg2rad(arr) print(x)</pre>	<pre>[1.57079633 3.14159265 4.71238898 6.28318531]</pre>
Find the angle of 1.0	<pre>import numpy as np x = np.arcsin(1.0) print(x)</pre>	<pre>1.5707963267948966</pre>
Property	Program	Output
Graph of sin(2*pi*x) (visualising the computation)	<pre>import numpy as np import matplotlib.pyplot as plt x = np.linspace(0, 1, 200) plt.plot(x, np.sin(2 * np.pi * x)) plt.title("y = sin(2*pi*x)"); plt.xlabel("x"); plt.ylabel("y"); plt.grid(True) plt.show()</pre>	

Property	Program	Output
		

d) Import a CSV file and perform various Statistical and Comparison operations on rows/columns

Property	Program	Output
Create a NumPy array with the same number of rows and columns as the CSV	<pre>import csv import numpy as np with open("winequality-red.csv", 'r') as f: wines = list(csv.reader(f, delimiter=";")) wines = np.array(wines[1:], dtype=float) print(wines.shape) wines</pre>	<pre>(1599, 12) array([[7.4 , 0.7 , 0. , ..., 0.56, 9.4 , 5.], [7.8 , 0.88, 0. , ..., 0.68, 9.8 , 5.], [7.3 , 0.45, 0.07, ..., 0.75, 11.2 , 7.], ..., [10.2 , 0.53, 0.77, ..., 0.62, 10.7 , 7.], [6.8 , 0.87, 0. , ..., 0.85, 10.7 , 5.], [8.9 , 0.48, 0.16, ..., 0.52, 12.1 , 6.]], shape=(1599, 12))</pre>
Statistical operations: mean / median of a column	<pre>import pandas as pd df = pd.read_csv('winequality-red.csv', delimiter=';') print('fixed acidity mean =', df['fixed acidity'].mean()) print('fixed acidity median =', df['fixed acidity'].median()) print('alcohol mean (NumPy) =', wines[:, 10].mean()) print('alcohol std (NumPy) =', wines[:, 10].std())</pre>	<pre>fixed acidity mean = 8.297811131957474 fixed acidity median = 8.3 alcohol mean (NumPy) = 10.378298936835524 alcohol std (NumPy) = 1.0379891601688396</pre>
Comparison: which wines have a quality rating higher than 5	<pre>wines[:, 11] > 5</pre>	<pre>array([False, False, True, ..., True, False, True], shape=(1599,))</pre>
Comparison: do any wines have a quality rating equal to 10	<pre>wines[:, 11] == 10</pre>	<pre>array([False, False, False, ..., False, False, False], shape=(1599,))</pre>
Select rows in wines where the quality is over 7	<pre>high_quality = wines[:, 11] > 7 wines[high_quality, :][:3, :]</pre>	<pre>array([[9.3 , 0.49 , 0.45 , 3. , 0.083 , 1. , 72. , 0.998 , 3.35 , 0.77 , 12.7 , 8.], [9.1 , 0.51 , 0.29 , 2.3 , 0.115 , 1. , 50. , 0.99967, 3.5 , 0.88 , 10.8 , 8.], [8.6 , 0.68 , 0.47 , 3.2 , 0.108 , 1. , 34. , 0.99868, 2.98 , 0.56 , 11.3 , 8.]])</pre>
Property	Program	Output
Look for wines with a lot of alcohol and high quality	<pre>high_quality_and_alcohol = (wines[:, 10] > 10) & (wines[:, 11] > 7) wines[high_quality_and_alcohol, 10:]</pre>	<pre>array([[12.7, 8.], [10.8, 8.], [11.3, 8.], [11.5, 8.], [11.7, 8.], [11.8, 8.], [12.6, 8.],</pre>

Property	Program	Output
		<pre> [10.5, 8.], [11.9, 8.], [10.2, 8.], [12.1, 8.], [12.4, 8.], [11.3, 8.], [11.7, 8.], [13.8, 8.], [11.4, 8.], [12.6, 8.], [12.4, 8.], [12.1, 8.], [12.3, 8.], [13. , 8.], [13.4, 8.], [11.9, 8.], [10.4, 8.], [12.2, 8.], [12.5, 8.], [10.3, 8.], [13.3, 8.], [11.4, 8.], [11.7, 8.], [11. , 8.], [12.8, 8.], [10.9, 8.], [10.7, 8.], [11.2, 8.] </pre>

e) Load an image file and do crop and flip operations using NumPy indexing

Property	Program	Output
Loading the image	<pre> import imageio.v2 as iio import matplotlib.pyplot as plt img = iio.imread("emma_stone.jpg") print(img.shape) plt.imshow(img); plt.title("Original image"); plt.axis("off") plt.show() </pre>	(427, 640, 3) Original image 
Cropping the image using NumPy slicing (rows 100:300, columns 200:450)	<pre> cropped = img[100:300, 200:450] print(cropped.shape) plt.imshow(cropped); plt.title("Cropped image"); plt.axis("off") plt.show() </pre>	(200, 250, 3) Cropped image 
Property	Program	Output
Flipping the image: vertical flip with a swap loop, then using slicing	<pre> img0 = img.copy() for i in range(img0.shape[0] // 2): c = img0[i, :, :].copy() img0[i, :, :] = img0[img0.shape[0] - i - 1, :, :] img0[img0.shape[0] - i - 1, :, :] = c fig, ax = plt.subplots(1, 3, figsize=(12, 3.6)) </pre>	

Property	Program	Output	
	<pre>ax[0].imshow(img0); ax[0].set_title("Vertical flip (loop)") ax[1].imshow(img[::-1]); ax[1].set_title("Vertical flip img[::-1]") ax[2].imshow(img[:, ::-1]); ax[2].set_title("Horizontal flip img[:, ::-1]") for a in ax: a.axis("off") plt.show()</pre>		
	Vertical flip (loop) 	Vertical flip img[::-1] 	Horizontal flip img[:, ::-1] 